Introduction to Arrays

An **array** is a collection of elements of the **same data type**, stored in contiguous memory locations and identified by a common name.

Suppose marks of five students are to be stored. Instead of declaring:

```
int m1, m2, m3, m4, m5;
```

we can declare:

```
int marks[5];
```

Thus, an array makes it easier to store and process a large number of similar values.

An array is a collection of elements of the same data type stored in contiguous memory locations and accessed using a common name with an index or subscript.

Main Characteristics

- All elements belong to the same data type.
- Elements are stored in contiguous memory locations.
- A single name represents the complete array.
- Individual elements are accessed through indices.
- Array indexing in C starts from **0**.
- Arrays can be processed efficiently using loops.

Array Notation and Representation

An individual array element is represented by writing the array name followed by its index inside square brackets.

General Notation

```
array_name[index]
```

For example:

```
marks[0]
marks[1]
marks[2]
```

If:

```
int marks[5] = {70, 80, 60, 90, 75};
```

then:

Index	Array Element	Value
0	`marks[0]`	70
1	`marks[1]`	80
2	`marks[2]`	60
3	`marks[3]`	90
4	`marks[4]`	75

Important Rule

For an array containing **n elements**, valid indices are:

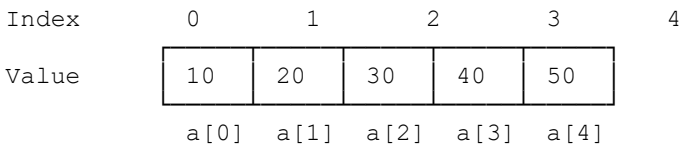
0 to n − 1

Representation of an Array in Memory

Array elements are stored in **contiguous memory locations**.

Suppose: `int a[5] = {10, 20, 30, 40, 50};`

Conceptually:



Because the elements are contiguous, the address of an element can conceptually be calculated as:

Address of `a[i]` = Base Address + i × size of each element

This property is one reason array elements can be accessed efficiently.

Declaring a One-Dimensional Array

A **one-dimensional array** is a linear collection of elements accessed using one index.

Syntax

```
data_type array_name[size];
```

Examples:

```
int marks[5];
float salary[10];
char grade[20];
```

In:

```
int marks[5];
```

`int` → Data type of elements
`marks` → Array name
`5` → Number of elements

The elements are:

```
marks[0]
marks[1]
marks[2]
marks[3]
marks[4]
```

Examples of One-Dimensional Arrays

```
int age[50];
float price[20];
double salary[100];
char grade[10];
```

One-dimensional arrays are useful for storing lists such as marks, salaries, prices, ages, sales figures, etc.

Initializing Arrays

Initialization means assigning initial values to array elements.

1. Initialization at Declaration

```
int a[5] = {10, 20, 30, 40, 50};
```

This creates:

```
a[0] = 10
a[1] = 20
a[2] = 30
a[3] = 40
a[4] = 50
```

2. Initialization Without Specifying Size

When all initial values are supplied, the size may be omitted:

```
int a[] = {10, 20, 30, 40, 50};
```

The compiler determines that the array contains five elements.

3. Partial Initialization

```
int a[5] = {10, 20};
```

The remaining elements are initialized to zero:

```
a[0] = 10
a[1] = 20
a[2] = 0
a[3] = 0
a[4] = 0
```

Another useful example:

```
int a[5] = {0};
```

initializes all five elements to zero.

Accessing Array Elements

An array element is accessed using its index.

Example:

```
int a[5] = {10, 20, 30, 40, 50};

printf("%d", a[2]);
```

Output:

```
30
```

because:

```
a[0] = 10
a[1] = 20
a[2] = 30
```

Changing an Individual Element

```
a[2] = 100;
```

Now the array becomes:

```
10 20 100 40 50
```

Accessing Elements Using a Loop

```
int i;

for (i = 0; i < 5; i++)
{
    printf("%d ", a[i]);
}
```

A loop is one of the most convenient methods for accessing all array elements.

Input and Output of Array Elements

Example Program

```
#include <stdio.h>

int main(void)
{
    int a[5];
    int i;

    printf("Enter 5 numbers:\n");

    for (i = 0; i < 5; i++)
    {
        scanf("%d", &a[i]);
    }

    printf("Array elements are:\n");

    for (i = 0; i < 5; i++)
    {
        printf("%d ", a[i]);
    }
```

```
    return 0;
}
```

If input is:

```
10 20 30 40 50
```

output is:

```
10 20 30 40 50
```

Manipulating Array Elements

Manipulation of array elements means performing operations on the values stored in an array.

Common operations include:

- Changing elements
- Addition and calculation of total
- Finding average
- Searching
- Finding maximum or minimum
- Sorting
- Copying elements

Example: Finding Sum of Array Elements

```
#include <stdio.h>

int main(void)
{
    int a[5] = {10, 20, 30, 40, 50};
    int i, sum = 0;

    for (i = 0; i < 5; i++)
    {
        sum += a[i];
    }

    printf("Sum = %d\n", sum);

    return 0;
}
```

Output:

```
Sum = 150
```

Example: Finding Largest Element

```
#include <stdio.h>

int main(void)
{
    int a[5] = {25, 60, 15, 80, 40};
    int i, largest;

    largest = a[0];
```

```
    for (i = 1; i < 5; i++)
    {
        if (a[i] > largest)
        {
            largest = a[i];
        }
    }

    printf("Largest = %d\n", largest);

    return 0;
}
```

Output:

```
Largest = 80
```

These programs demonstrate how individual elements can be processed through loops.

Arrays of Unknown or Varying Size

This syllabus point requires a little care because C provides different ways of dealing with array sizes that are not fixed as a literal constant in the source code.

Array Size Determined from Initialization

The size can be omitted when an initializer is provided:

```
int a[] = {10, 20, 30, 40, 50};
```

The compiler determines the size as 5.

This is an example where the size is not explicitly written.

Variable Length Arrays

Example:

```
#include <stdio.h>

int main(void)
{
    int n, i;

    printf("Enter size: ");
    scanf("%d", &n);

    int a[n];

    for (i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }

    for (i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }
```

```
    return 0;
}
```

Here, the size `n` is supplied during program execution.

Important Note

Variable Length Arrays were introduced in **C99**, but support is optional in later C standards and some compilers do not support them. For truly dynamic storage, C programmers commonly use **dynamic memory allocation with pointers**

Two-Dimensional Arrays

A **two-dimensional array** represents elements in rows and columns.

It is useful for representing:

- Matrices
- Tables
- Student marks across subjects
- Tabular numerical information

A two-dimensional array is an array whose elements are logically arranged in rows and columns and are accessed using two indices.

Declaration

Syntax: `data_type array_name[rows][columns];`

Example: `int a[2][3];`

It contains:

2 × 3 = 6 elements

represented as:

```
          Column
          0      1      2

Row 0   a[0][0] a[0][1] a[0][2]

Row 1   a[1][0] a[1][1] a[1][2]
```

Initializing a Two-Dimensional Array

```
int a[2][3] =
{
    {10, 20, 30},
    {40, 50, 60}
};
```

It represents:

	Column 0	Column 1	Column 2

Row 0	10	20	30
Row 1	40	50	60

Therefore:

```
a[0][0] = 10
a[0][1] = 20
a[0][2] = 30
a[1][0] = 40
a[1][1] = 50
a[1][2] = 60
```

Accessing Two-Dimensional Array Elements

An individual element requires two indices:

```
a[row][column]
```

For example:

```
printf("%d", a[1][2]);
```

prints: `60`

Nested loops are normally used to process a two-dimensional array.

Input and Output of a Two-Dimensional Array

```
#include <stdio.h>

int main(void)
{
    int a[2][3];
    int i, j;

    printf("Enter 6 elements:\n");

    for (i = 0; i < 2; i++)
    {
        for (j = 0; j < 3; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }

    printf("Matrix is:\n");

    for (i = 0; i < 2; i++)
    {
        for (j = 0; j < 3; j++)
        {
            printf("%d ", a[i][j]);
        }

        printf("\n");
    }

    return 0;
}
```

If input is:

```
10 20 30
40 50 60
```

output is:

```
10 20 30
40 50 60
```

Important Concept

In a 2-D array:

Outer loop → Rows

Inner loop → Columns

This is why your previous topic of **nested loops** is directly connected with two-dimensional arrays.

Multidimensional Arrays

An array having two or more dimensions is called a **multidimensional array**. In common introductory usage, the term often refers particularly to arrays of three or more dimensions after 2-D arrays have been discussed separately.

Examples:

```
int a[2][3];
```

is two-dimensional.

```
int a[2][3][4];
```

is three-dimensional.

Three-Dimensional Array

Syntax:

```
data_type array_name[size1][size2][size3];
```

Example:

```
int data[2][3][4];
```

The total number of elements is:

2 × 3 × 4 = 24

An individual element can be accessed as:

```
data[1][2][3]
```

Initialization Example

```
int a[2][2][2] =
{
    {
        {1, 2},
        {3, 4}
    },
    {
        {5, 6},
        {7, 8}
    }
};
```

Three nested loops can be used to process a three-dimensional array.

One-Dimensional vs Two-Dimensional vs Multidimensional Arrays

Feature	1-D	2-D	3-D Example
Representation	List	Rows and columns	Multiple 2-D layers
Indices	One	Two	Three
Example	`a[5]`	`a[2][3]`	`a[2][3][4]`
Common processing	Single loop	Two nested loops	Three nested loops
Example use	Marks	Matrix	Multi-index data

Advantages of Arrays

1. A large number of similar values can be stored under one name.
2. Array elements can be processed easily using loops.
3. Code becomes shorter and more systematic.
4. Elements can be accessed directly through indices.
5. Arrays are useful for sorting and searching.
6. Two-dimensional arrays make matrix and tabular processing convenient.
7. Arrays form the basis for many more advanced data structures and programming techniques.

Limitations

1. Array elements must have compatible element types—an array itself has one element type.
2. Ordinary fixed-size arrays cannot be resized after creation.
3. C does not automatically check array bounds.
4. Inserting/deleting elements in an ordered sequence may require shifting elements.
5. Incorrect indexing can cause undefined behavior.

Introduction to Pointers

A **pointer** is a variable that stores the memory address of another variable.

Normally, a variable stores a value:

```
int a = 10;
```

Here, `a` stores the value `10`.

A pointer, on the other hand, can store the address of `a`:

```
int *p;
p = &a;
```

Here:

- `a` stores `10`
- `&a` represents the address of `a`
- `p` stores the address of `a`
- `*p` gives the value stored at that address

A pointer is a variable that stores the memory address of another variable or object.

Simple Representation

Suppose:

```
int a = 10;
int *p = &a;
```

Conceptually:

```
Variable a
Address: 1000
Value:   10

Variable p
Value:   1000
```

Therefore:

```
a   = 10
&a  = address of a
p   = address of a
*p  = 10
```

Pointers are one of the most powerful features of C programming.

Characteristics of Pointers

Important characteristics of pointers are:

1. A pointer stores a **memory address**.
2. A pointer has a type associated with it.
3. A pointer can point to a variable of a compatible type.
4. The address operator `&` is commonly used to obtain the address of an object.
5. The indirection operator `*` can be used to access the object pointed to by a pointer.
6. Pointer values can participate in certain arithmetic operations.
7. Pointers can be passed to functions.
8. Pointers can be used to modify variables in calling functions.
9. Arrays and pointers are closely related in C.
10. Pointers are important in dynamic memory allocation, structures, strings, and data structures.

Advantages of Pointers

Pointers are useful because they:

- Allow efficient access to memory.
- Enable functions to modify caller variables.
- Facilitate dynamic memory allocation.
- Support efficient array and string processing.
- Help create linked lists, trees, and other dynamic structures.
- Allow functions themselves to be referenced through function pointers.

The `&` and `*` Operators

Two operators are especially important in pointer programming:

`&` — Address Operator

`*` — Indirection/Dereference Operator

Address Operator `&`

The `&` operator returns the address of an object.

Example:

```
int a = 10;

printf("%p", (void *)&a);
```

`&a` means:

Address of variable `a`

If `a` is stored at address 1000, then conceptually:

```
&a = 1000
```

Indirection Operator *

When used with a pointer expression, `*` accesses the object stored at the address represented by that pointer.

Example:

```
int a = 10;
int *p = &a;

printf("%d", *p);
```

Output:

```
10
```

Because `p` contains the address of `a`, `*p` accesses the value of `a`.

Relationship

```
a  → value of a
&a → address of a
```

```
p  → address stored in pointer
*p → object/value pointed to by p
```

Example

```
#include <stdio.h>

int main(void)
{
    int a = 25;
    int *p = &a;

    printf("Value of a = %d\n", a);
    printf("Value through pointer = %d\n", *p);

    return 0;
}
```

Output:

```
Value of a = 25
Value through pointer = 25
```

Pointer Type Declaration

A pointer must be declared before use.

Syntax

```
data_type *pointer_name;
```

Examples:

```
int *p;
float *q;
char *ch;
double *d;
```

Here: `int *p;`

means that `p` is a pointer intended to point to an `int`.

Similarly: `float *q;`

means that `q` points to a `float`.

Pointer Assignment

After declaring a pointer, the address of a compatible object can be assigned to it.

Example:

```
int a = 10;
int *p;

p = &a;
```

Now:

```
p = address of a
*p = 10
```

Declaration and assignment may also be combined:

```
int a = 10;
int *p = &a;
```

Complete Example

```
#include <stdio.h>

int main(void)
{
    int a = 50;
    int *p;

    p = &a;

    printf("Value of a = %d\n", a);
    printf("Value through pointer = %d\n", *p);

    return 0;
}
```

Output:

```
Value of a = 50
Value through pointer = 50
```

Pointer Arithmetic

C allows certain arithmetic operations on pointers, especially when pointers refer to elements of an array.

Common pointer operations include:

- Increment `p++`
- Decrement `p--`
- Addition with an integer
- Subtraction with an integer
- Difference between pointers within the same array
- Pointer comparisons within appropriate contexts

Example

```
int a[5] = {10, 20, 30, 40, 50};
int *p = a;
```

Initially, `p` points to:

```
a[0]
```

If we write:

```
p++;
```

it points to:

`a[1]`

The pointer advances by the size of the pointed-to type, not necessarily by one byte.

Example of Pointer Arithmetic

```
#include <stdio.h>

int main(void)
{
    int a[5] = {10, 20, 30, 40, 50};
    int *p = a;
    int i;

    for (i = 0; i < 5; i++)
    {
        printf("%d ", *p);
        p++;
    }

    return 0;
}
```

Output:

```
10 20 30 40 50
```

Pointer Addition

If: `int *p = a;`

then: `p + 1`

points to the next integer element.

Conceptually:

```
p     → a[0]
p + 1 → a[1]
p + 2 → a[2]
```

And:

```
*(p + 2)
```

accesses `a[2]`.

Important Rule

Pointer arithmetic is meaningful primarily when the pointer remains within the same array object or one position past its last element, according to C's pointer rules.

Passing Pointers to Functions

A pointer can be passed as an argument to a function.

Syntax

```
void function(int *p)
{
    /* statements */
}
```

Calling:

```
function(&a);
```

The function receives the address of `a`.

Example

```
#include <stdio.h>

void doubleValue(int *p)
{
    *p = *p * 2;
}

int main(void)
{
    int a = 10;

    doubleValue(&a);

    printf("Value = %d\n", a);

    return 0;
}
```

Output:

```
Value = 20
```

Advantages

Passing pointers to functions:

- Allows modification of original variables.
- Can avoid copying large objects.
- Enables a function to produce multiple results through output parameters.
- Is widely used with arrays and structures.

Swapping Two Numbers Using Pointers

This is an important examination program.

```
#include <stdio.h>

void swap(int *x, int *y)
{
    int temp;

    temp = *x;
    *x = *y;
    *y = temp;
```

```
}

int main(void)
{
    int a = 10;
    int b = 20;

    swap(&a, &b);

    printf("a = %d\n", a);
    printf("b = %d\n", b);

    return 0;
}
```

Output:

```
a = 20
b = 10
```

This program clearly demonstrates pointer-based call-by-reference behavior.

Array of Pointers

An **array of pointers** is an array whose elements are pointer values.

Syntax

```
data_type *array_name[size];
```

Example:

```
int *p[5];
```

Here, `p` is an array containing five pointers to `int`.

Example

```
int a = 10;
int b = 20;
int c = 30;

int *p[3] = {&a, &b, &c};
```

Then:

```
p[0] → address of a
p[1] → address of b
p[2] → address of c
```

And:

```
*p[0]
```

gives `10`.

Program

```
#include <stdio.h>

int main(void)
{
    int a = 10, b = 20, c = 30;
    int *p[3] = {&a, &b, &c};
    int i;

    for (i = 0; i < 3; i++)
    {
        printf("%d ", *p[i]);
    }

    return 0;
}
```

Output:

```
10 20 30
```

Uses of Array of Pointers

Arrays of pointers are useful for:

- Storing addresses of multiple variables.
- Handling arrays of strings.
- Dynamic data structures.
- Tables of dynamically allocated objects.
- Efficient sorting or rearrangement by changing pointer order instead of moving large data objects.

Pointer to Function

Just as a pointer can store the address of a variable, C also allows a pointer to refer to a **function**.

A **function pointer** stores the address of a function with a compatible function type.

Syntax

```
return_type (*pointer_name)(parameter_types);
```

Example:

```
int (*p)(int, int);
```

This declares `p` as a pointer to a function that:

- accepts two `int` arguments
- returns an `int`

Example of Function Pointer

```
#include <stdio.h>

int add(int a, int b)
{
    return a + b;
```

```
}

int main(void)
{
    int (*p)(int, int);

    p = add;

    printf("%d\n", p(10, 20));

    return 0;
}
```

Output:

```
30
```

Here:

```
p = add;
```

assigns the function's address to the function pointer.

Then:

```
p(10, 20)
```

calls the function through the pointer.

Uses of Function Pointers

Function pointers are useful in:

- Callback mechanisms
- Menu-driven programs
- Selection of functions at run time
- Generalized algorithms
- Function tables

Pointer to Pointer

A **pointer to pointer** is a pointer that stores the address of another pointer.

It is also known as a **double pointer**.

Declaration

```
int **pp;
```

Here:

- `pp` is a pointer to an `int *`
- it can store the address of a pointer to `int`

Example:

```
int a = 10;
int *p = &a;
int **pp = &p;
```

Representation:

```
a  = 10
p  = address of a
pp = address of p
```

Accessing Value Through Pointer to Pointer

Given:

```
int a = 10;
int *p = &a;
int **pp = &p;
```

then:

```
a    → 10
*p   → 10
**pp → 10
```

Program

```
#include <stdio.h>

int main(void)
{
    int a = 10;
    int *p = &a;
    int **pp = &p;

    printf("a = %d\n", a);
    printf("*p = %d\n", *p);
    printf("**pp = %d\n", **pp);

    return 0;
}
```

Output:

```
a = 10
*p = 10
**pp = 10
```

Uses of Pointer to Pointer

Pointer-to-pointer variables are useful for:

- Modifying a pointer inside a function.
- Working with dynamic two-dimensional structures.
- Handling arrays of pointers.
- Managing dynamic memory.
- Implementing complex data structures.
- Processing command-line argument arrays.

Common Pointer Errors

Pointers are powerful but must be used carefully.

1. Uninitialized Pointer

```
int *p;
*p = 10;
```

This is unsafe because `p` does not yet point to a valid object.

2. Dereferencing a Null Pointer

```
int *p = NULL;
printf("%d", *p);
```

A null pointer must not be dereferenced.

3. Invalid Pointer Arithmetic

A pointer should not be moved arbitrarily outside the valid bounds of its related array object.

4. Dangling Pointer

A **dangling pointer** refers to storage whose lifetime has already ended or whose dynamically allocated memory has been freed.

These errors can cause undefined behavior.

Pointer vs Ordinary Variable

Ordinary Variable	Pointer Variable
Stores a data value	Stores an address
Example: `int a = 10;`	Example: `int *p = &a;`
Access value using `a`	Access pointed object using `*p`
`&a` gives its address	`p` itself holds an address

Important Complete Example

```
#include <stdio.h>

void change(int *p);

int main(void)
{
    int a = 10;
    int *ptr = &a;

    printf("Original value = %d\n", a);
    printf("Value using pointer = %d\n", *ptr);

    change(&a);

    printf("Changed value = %d\n", a);

    return 0;
```

```
}

void change(int *p)
{
    *p = 50;
}
```

Output:

```
Original value = 10
Value using pointer = 10
Changed value = 50
```

This program covers:

Pointer declaration + `&` operator + `*` operator + pointer assignment + passing pointer to function + reference-style modification

Introduction to String

A **string** is a sequence of characters stored in a character array and terminated by a special character called the **null character (`'\0'`)**.

In C, there is no separate built-in `string` data type. Strings are represented using **arrays of characters**.

Example

```
char name[10] = "Rahul";
```

Memory representation:

```
R   a   h   u   l   \0
```

Here:

- `R, a, h, u, l` are characters.
- `'\0'` indicates the **end of the string**.
- `'\0'` occupies one character of memory.

Important Point

The string `"Hello"` contains **5 visible characters**, but the character array required to store it as a C string needs **6 positions** because of `'\0'`.

```
char str[6] = "Hello";
```

Initializing a String

A string can be initialized in several ways.

Method 1: Using a String Literal

```
char str[] = "Hello";
```

The compiler automatically allocates sufficient space.

It is equivalent to:

```
char str[] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

Method 2: Specifying Array Size

```
char str[10] = "Hello";
```

The first six positions contain:

```
H e l l o \0
```

The remaining positions are unused/initialized to zero.

Method 3: Character-by-Character Initialization

```
char str[] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

Here, the null character must be explicitly included.

Important

```
char str[] = {'H', 'e', 'l', 'l', 'o'};
```

is a **character array**, but it is not a properly terminated C string because `'\0'` is missing.

Accessing String Elements

Individual characters of a string can be accessed using an **index**.

The index of an array starts from `0`.

Example

```
#include <stdio.h>

int main()
{
    char str[] = "Computer";

    printf("%c", str[0]);
    printf("%c", str[3]);

    return 0;
}
```

Output

```
Cp
```

The string is:

```
Index:   0 1 2 3 4 5 6 7
         C o m p u t e r
```

Therefore:

```
str[0] = 'C'
str[1] = 'o'
str[2] = 'm'
str[3] = 'p'
```

Modifying String Elements

Since a character array is modifiable, individual characters can be changed.

```
char str[] = "Hello";

str[0] = 'J';

printf("%s", str);
```

Output

```
Jello
```

Reading and Printing Strings

Using `scanf()`

```
char name[20];

scanf("%s", name);
```

Example input:

```
Rahul
```

`scanf("%s", name)` stops reading when it encounters whitespace.

Therefore, it cannot normally read a complete sentence containing spaces.

Using `printf()`

```
printf("%s", name);
```

`%s` is the format specifier used for strings.

Using `fgets()`

For reading a line, `fgets()` is preferred:

```
char str[50];

fgets(str, sizeof(str), stdin);
```

It can read spaces as part of the input.

Array of Strings

An **array of strings** is used to store multiple strings.

In C, it is generally represented as a **two-dimensional character array**.

Syntax

```
char array_name[number_of_strings][maximum_string_length];
```

Example

```
char names[3][20] = {
    "Rahul",
    "Amit",
    "Priya"
};
```

Here:

- `3` = number of strings
- `20` = maximum storage size for each string

Conceptually:

```
Rahul
Amit
Priya
```

Accessing an Array of Strings

```
printf("%s", names[0]);
```

Output:

```
Rahul
```

To print all strings:

```
#include <stdio.h>

int main()
{
    char names[3][20] = {"Rahul", "Amit", "Priya"};

    for(int i = 0; i < 3; i++)
    {
        printf("%s\n", names[i]);
    }

    return 0;
}
```

Output

```
Rahul
Amit
Priya
```

Passing String to a Function

A string can be passed to a function by passing the **character array** to the function.

Example

```
#include <stdio.h>

void display(char str[])
{
    printf("%s", str);
}

int main()
{
    char name[] = "Rahul";

    display(name);

    return 0;
}
```

Output

```
Rahul
```

The function parameter:

```
char str[]
```

can also be written as:

```
char *str
```

Therefore:

```
void display(char str[])
```

and

```
void display(char *str)
```

are commonly used equivalently for receiving a string in a function.

Example: Passing String and Finding Length

```
#include <stdio.h>
#include <string.h>

void length(char str[])
{
    printf("Length = %lu", strlen(str));
}

int main()
{
    char name[] = "Computer";

    length(name);
```

```
    return 0;
}
```

Output

```
Length = 8
```

String Functions in C

String handling functions are provided by the header file:

```
#include <string.h>
```

Some important string functions are:

Function	Purpose
`strlen()`	Finds length of a string
`strcpy()`	Copies one string to another
`strcat()`	Concatenates two strings
`strcmp()`	Compares two strings
`strchr()`	Searches for a character
`strstr()`	Searches for a substring

`strlen()` Function

The `strlen()` function returns the number of characters in a string, excluding the null character `'\0'`.

Syntax

```
strlen(string);
```

Example

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str[] = "Computer";

    printf("%lu", strlen(str));

    return 0;
}
```

Output

```
8
```

Important

For:

```
char str[] = "Hello";
```

`strlen(str)` returns:

```
5
```

It does **not** count `'\0'`.

`strcpy()` Function

The `strcpy()` function copies one string into another.

Syntax

```
strcpy(destination, source);
```

Example

```
#include <stdio.h>
#include <string.h>

int main()
{
    char source[] = "Hello";
    char destination[20];

    strcpy(destination, source);

    printf("%s", destination);

    return 0;
}
```

Output

```
Hello
```

Here:

```
source       → Hello
destination  → Hello
```

Important

The destination array must have enough space to store the copied string including `'\0'`.

`strcat()` Function

The `strcat()` function is used to **concatenate**, or join, two strings.

Syntax

```
strcat(destination, source);
```

The source string is appended to the end of the destination string.

Example

```
#include <stdio.h>
```

```
#include <string.h>

int main()
{
    char str1[20] = "Hello ";
    char str2[] = "World";
    strcat(str1, str2);
    printf("%s", str1);
    return 0;
}
```

Output

```
Hello World
```

Before:

```
str1 = "Hello "
str2 = "World"
```

After:

```
str1 = "Hello World"
```

`strcmp()` Function

The `strcmp()` function compares two strings lexicographically.

Syntax

```
strcmp(string1, string2);
```

Return values

Return value	Meaning
`0`	Both strings are equal
`< 0`	First string is smaller
`> 0`	First string is greater

Example

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str1[] = "Apple";
    char str2[] = "Apple";
    if(strcmp(str1, str2) == 0)
        printf("Strings are equal");
    else
        printf("Strings are not equal");
    return 0;
}
```

Output: `Strings are equal`

Important Exam Point

Do not compare C strings using:

```
str1 == str2
```

For string contents, use:

```
strcmp(str1, str2)
```

`strchr()` Function

The `strchr()` function searches for the **first occurrence of a character** in a string.

Syntax

```
strchr(string, character);
```

Example

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str[] = "Computer";
    printf("%s", strchr(str, 'p'));
    return 0;
}
```

The first `p` occurs in `"puter"`.

Output: `puter`

`strstr()` Function

The `strstr()` function searches for the first occurrence of a **substring** in another string.

Syntax

```
strstr(string, substring);
```

Example

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str[] = "I am studying Computer Application";
    printf("%s", strstr(str, "Computer"));
    return 0;
}
```

Output: `Computer Application`